Trabalho Prático 1
A Forja do Blibblon
Data de Entrega: 19/10/2026

1 Descrição do problema

Blibblon Trombaferro é primo de quinto grau de um herói anão extremamente famoso, cujo nome nossos advogados da Terra Média nos aconselharam vivamente a não mencionar. Blibblon nunca destruiu anel nenhum, nunca desceu a mina nenhuma e nunca enfrentou dragão nenhum, mas herdou o sobrenome, o que na economia de aventura contemporânea vale mais do que a biografia. Aproveitando o parentesco, abriu uma forja e passou a atender aventureiros que querem montar a build perfeita.

O negócio funciona assim. Blibblon mantém um catálogo grande de itens, e cada item concede alguns modificadores de atributo. O ponto é que **cada item concede pouquíssimos atributos**: uma Barba Postiça de Prestígio pode conceder `carisma +7` e `furtividade -3` e mais nada, ainda que o catálogo de atributos tenha dezenas de entradas. Armazenar isso como uma tabela completa de itens por atributos desperdiça memória proporcional ao produto das duas dimensões, com quase todas as posições em zero.

Aventureiros encomendam itens. Como Blibblon é um anão só e a fama do primo atrai mais clientes do que a competência dele comporta, as encomendas entram numa **fila de espera** e são forjadas na ordem em que chegaram. Quando um item fica pronto, é equipado no aventureiro que o encomendou. Aventureiros também mudam de ideia, e a operação de desequipar remove sempre a **última peça equipada** — a build é montada e desmontada pela ponta.

Além disso, Blibblon vende consultoria. Um aventureiro pode perguntar quais itens do catálogo mais valeriam a pena para ele, segundo um subconjunto de métricas previamente definido. A ideia central é a mesma de qualquer sistema de recomendação: **cada métrica induz, no instante da consulta, uma ordenação global de todo o catálogo**; as ordenações das métricas escolhidas são combinadas por uma regra de pontuação ponderada, também global; e só então o resultado é filtrado pelo que aquele aventureiro ainda não possui. A ordenação é global, a resposta é local.

O objetivo do trabalho é implementar esse sistema, sustentando quatro classes de operação: manutenção do catálogo de modificadores, gestão da fila de encomendas, montagem e desmontagem de builds, e consultas de ranking sobre o catálogo inteiro.

Cada atributo possui:

```
Atributo { id, nome }
```

Cada item possui:

```
Item { id, nome }
```

Cada aventureiro possui:

```
Aventureiro { id, nome }
```

Os identificadores são inteiros não negativos, atribuídos automaticamente em ordem crescente a partir de 0, considerando apenas as entidades já cadastradas de cada tipo. Os nomes não são necessariamente únicos: dois itens distintos podem ter o mesmo nome, e o mesmo vale para atributos e aventureiros.

A tabela de modificadores é a matriz

`modificador[id_item][id_atributo]`

em que uma célula existe se e somente se o modificador é diferente de zero. Modificadores podem ser **negativos**: um item pode conceder poder às custas de algum prejuízo, e a Barba Postiça de Prestígio é o exemplo canônico disso. **Atribuir o valor zero a uma célula equivale a removê-la**, e a estrutura deve refletir isso. Essa regra não é decorativa: ela obriga a representação escolhida a suportar remoção, e não apenas inserção e atualização.
Todos os valores do sistema são determinísticos. Nenhuma operação envolve sorteio, probabilidade ou variação aleatória: a mesma entrada produz sempre exatamente a mesma saída.

1.1 Representação interna

O sistema deve permitir que a representação interna da matriz seja alterada durante a execução, sem alterar o estado lógico das informações já cadastradas. São obrigatórias duas representações:

- **modo L** — lista cruzada, isto é, células encadeadas simultaneamente por linha e por coluna, com vetores de cabeças de linha e de coluna;
- **modo T** — vetor de tripletas `(item, atributo, valor)` mantido ordenado, em memória contígua.

A migração entre modos deve preservar todas as células armazenadas e seus valores.

1.2 Métricas

Seja T o número de atributos cadastrados no instante da consulta e N o número de itens cadastrados. Seja $v_i(a)$ o modificador do item i no atributo a, com $v_i(a) = 0$ quando a célula não existe.
Seja $b_h(a)$ o **vetor da build** do aventureiro h, isto é, a soma dos modificadores no atributo a de todos os itens atualmente equipados por h. É exatamente o que a consulta LH imprime. Se um item estiver equipado mais de uma vez, contribui uma vez para cada ocorrência. As métricas suportadas pelo sistema são:

- **PODER**: soma de todos os modificadores do item, definida por

$$\text{PODER}(i) \;=\; \sum_{a=0}^{T-1} v_i(a).$$

- **EQUILIBRIO**: mede o quanto os modificadores do item estão distribuídos de forma uniforme entre os atributos, em vez de concentrados em um só, e é definida a partir do desvio dos modificadores, dada por

$$\text{EQUILIBRIO}(i) \;=\; \frac{1}{1+\sigma_i},$$

em que

$$\sigma_i = \sqrt{\frac{1}{T}\sum_{a=0}^{T-1}\big(v_i(a)-\mu_i\big)^2} \qquad \text{e} \qquad \mu_i = \frac{\text{PODER}(i)}{T}.$$

Note que as somas percorrem **todos os T atributos cadastrados**, incluindo aqueles em que o item não possui célula e portanto vale zero, e que o desvio é **populacional**, dividindo por T e não por $T-1$. Se $T = 0$, defina $\text{EQUILIBRIO}(i) = 1$.

- **SINERGIA**: mede o quanto o item combina com a build atual do aventureiro consultado, pela similaridade cosseno entre a linha do item e o vetor da build, dada por

$$\text{SINERGIA}(i,h) \;=\; \frac{\displaystyle\sum_{a=0}^{T-1} v_i(a)\, b_h(a)}{\sqrt{\displaystyle\sum_{a=0}^{T-1} v_i(a)^2}\;\cdot\;\sqrt{\displaystyle\sum_{a=0}^{T-1} b_h(a)^2}}.$$

  O valor pertence ao intervalo $[-1, 1]$: um valor negativo indica um item que trabalha contra a build. Se qualquer um dos dois fatores do denominador for zero — build vazia, ou item sem nenhum modificador — defina $\text{SINERGIA}(i,h) = 0$.

Cada métrica define, no instante em que é consultada, uma ordenação global dos itens. Para combinar múltiplas métricas, deve-se utilizar a seguinte **regra de pontuação por ordenação**: para um catálogo com N itens, o item na i-ésima posição de uma dada métrica recebe $N - i$ pontos nessa métrica, considerando posições indexadas a partir de 0. Esse valor é então multiplicado pelo peso associado à métrica na consulta. A pontuação final de um item é a soma das pontuações obtidas em todas as métricas da consulta. Dentro de cada métrica, o desempate é feito pelo identificador do item, em ordem crescente.
Ao ordenar por uma métrica, **compare os valores arredondados para 6 casas decimais**. Sem essa regra, dois itens matematicamente empatados poderiam ser separados por ruído de ponto flutuante, e a ordem deixaria de ser determinística.

1.3 Entrada e saída

O programa recebe como entrada um arquivo com 12 tipos de linha. A primeira linha da entrada é sempre do tipo E. Toda linha de entrada produz uma ou mais linhas de saída, sempre iniciadas pelo mesmo comando da entrada.

Modo de armazenamento:

```
E <modo>
```

em que <modo> pertence a {L, T}. Migra a matriz para a representação indicada, preservando o estado lógico. A saída é igual à entrada:

```
E <modo>
```

Cadastro de atributo:

```
A <nome>
```

O campo <nome> contém apenas letras e não possui espaços. A saída é:

```
A <id_atributo>
```

Cadastro de item:

```
I <nome>
```

O campo <nome> contém apenas letras e não possui espaços. A saída é:

```
I <id_item>
```

Definição de modificador:

```
M <id_item> <id_atributo> <valor>
```

Define o modificador que o item concede naquele atributo. O campo <valor> é um inteiro, podendo ser negativo. Se <valor> for zero, a célula correspondente deixa de existir. Se já houver um modificador definido para aquele par, ele é substituído. A saída é:

```
M <nome_item> <nome_atributo> <valor>
```

Cadastro de aventureiro:

```
H <nome>
```

O campo <nome> contém apenas letras e não possui espaços. A saída é:

```
H <id_aventureiro>
```

Encomenda:

```
F <id_aventureiro> <id_item>
```

Registra uma encomenda ao final da fila de espera da forja. A fila é única, global e compartilhada por todos os aventureiros, porque Blibblon é um só. A saída informa a posição da encomenda na fila, contada a partir de 1:

```
F <posicao>
```

Turnos de trabalho:

```
Z <k>
```

Blibblon trabalha <k> turnos, em que <k> é um inteiro estritamente positivo. Cada turno forja exatamente uma encomenda, retirada do início da fila, e equipa o item no aventureiro que a encomendou. Se a fila esvaziar antes de <k> turnos, os turnos restantes são perdidos.
Um mesmo item pode ser equipado mais de uma vez pelo mesmo aventureiro; nesse caso, seus modificadores contribuem tantas vezes quantas estiver equipado.
A primeira linha da saída é:

```
Z <n_forjadas>
```

Se n_forjadas for maior que zero, deve ser impressa, logo após a linha principal, uma segunda linha listando as encomendas atendidas na ordem em que foram forjadas:

```
encomenda_1 <id_aventureiro> <id_item> ... encomenda_n <id_aventureiro>
<id_item>
```

Se n_forjadas for zero, essa segunda linha não deve ser impressa.
Desequipar:

```
D <id_aventureiro>
```

Remove da build do aventureiro a última peça equipada, isto é, a mais recentemente forjada para ele. A saída é:

```
D <nome_aventureiro> <nome_item>
```

Se o aventureiro não tiver nenhum item equipado, a saída é:

```
D <nome_aventureiro> VAZIO
```

Consulta de item:

```
LI <id_item>
```

Lista os atributos modificados pelo item e os respectivos valores, em ordem crescente de identificador de atributo:

```
LI <nome_item> atributo_1 <id_a1> <valor_1> ... atributo_M <id_aM>
<valor_M>
```

Se o item não modificar nenhum atributo, a saída é:

```
LI <nome_item> VAZIO
```

Consulta de atributo:

```
LA <id_atributo>
```

Lista os itens que modificam o atributo e os respectivos valores, em ordem crescente de identificador de item:

```
LA <nome_atributo> item_1 <id_i1> <valor_1> ... item_M <id_iM> <valor_M>
```

Se nenhum item modificar o atributo, a saída é:

```
LA <nome_atributo> VAZIO
```

Consulta de build:

```
LH <id_aventureiro>
```

Imprime o vetor da build do aventureiro. Apenas atributos cujo total é diferente de zero são impressos, em ordem crescente de identificador de atributo:

```
LH <nome_aventureiro> atributo_1 <id_a1> <total_1> ... atributo_M <id_aM>
<total_M>
```

Se o aventureiro não tiver itens equipados, ou se todos os totais forem zero, a saída é:

```
LH <nome_aventureiro> VAZIO
```

Consulta de recomendação:

```
Q <id_aventureiro> <n> <m> <metrica_1> <peso_1> ... <metrica_m> <peso_m>
```

em que <n> e <m> são inteiros estritamente positivos e cada <metrica_j> pertence a {PODER, EQUILIBRIO, SINERGIA}. As métricas informadas em uma consulta são distintas entre si e os pesos são números reais positivos, fornecidos com exatamente duas casas decimais.
Calcule a pontuação final de todos os N itens cadastrados conforme a Seção 1.2. Em seguida, descarte os itens que o aventureiro tem atualmente equipados. Entre os itens restantes, retorne os <n> de maior pontuação, em ordem decrescente, com desempate pelo identificador do item, em ordem crescente. A saída contém uma linha por resultado:

```
Q <posicao> <id_item> <pontuacao>
```

em que <posicao> é contada a partir de 1 e o campo <pontuacao> deve ser impresso com exatamente 2 casas decimais. Caso haja menos itens candidatos do que <n>, devem ser listados apenas os existentes. Se não houver nenhum candidato, a consulta não produz nenhuma linha de saída.

1.4 Especificações da entrada

A entrada é sempre válida quanto ao formato dos comandos. Em particular:

- a primeira linha da entrada é sempre do tipo E;
- os comandos A, I e H são sempre válidos;
- os comandos M, F, D, LI, LA, LH e Q referenciam apenas entidades já cadastradas;
- os valores <k>, <n> e <m> são sempre inteiros estritamente positivos;
- os campos de nome contêm apenas letras e não possuem espaços;
- os pesos são números reais positivos com exatamente duas casas decimais.

Nenhuma operação falha por regra de negócio. As consultas podem não encontrar resultados, produzindo a saída VAZIO conforme especificado para cada comando, e o comando Z pode encontrar a fila vazia.

1.5 Exemplo de entrada e saída

```
E L
A Forca
A Carisma
A Furtividade
I MachadoGenerico
I BarbaPosticaDePrestigio
I BotasSilenciosas
I ElmoDoPrimo
M 0 0 5
M 1 1 7
M 1 2 -3
M 2 2 4
M 3 0 2
M 3 1 2
H Blibblon
H Grunilda
LI 1
LA 2
F 0 0
F 1 1
F 0 3
Z 2
LH 0
LH 1
Q 0 3 2 PODER 1.00 SINERGIA 3.00
D 0
LH 0
E T
Z 5
LH 0
Q 1 3 3 PODER 1.00 EQUILIBRIO 1.00 SINERGIA 1.00
```

Figura 1: Exemplo de entrada

```
E L
A 0
A 1
A 2
I 0
I 1
I 2
I 3
M MachadoGenerico Forca 5
M BarbaPosticaDePrestigio Carisma 7
M BarbaPosticaDePrestigio Furtividade -3
M BotasSilenciosas Furtividade 4
M ElmoDoPrimo Forca 2
M ElmoDoPrimo Carisma 2
H 0
H 1
LI BarbaPosticaDePrestigio atributo_1 1 7 atributo_2 2 -3
LA Furtividade item_1 1 -3 item_2 2 4
F 1
F 2
F 3
Z 2
encomenda_1 0 0 encomenda_2 1 1
LH Blibblon atributo_1 0 5
LH Grunilda atributo_1 1 7 atributo_2 2 -3
Q 1 3 10.00
Q 2 1 9.00
Q 3 2 5.00
D Blibblon MachadoGenerico
LH Blibblon VAZIO
E T
Z 1
encomenda_1 0 3
LH Blibblon atributo_1 0 2 atributo_2 1 2
Q 1 0 8.00
Q 2 3 8.00
Q 3 2 6.00
```

Figura 2: Exemplo de saída

Detalhamento da primeira consulta. No instante da consulta há $N = 4$ itens e $T = 3$ atributos. Blibblon tem apenas o Machado Genérico equipado, então seu vetor de build é `forca` $= 5$.

Ordenando por PODER, os valores são $5, 4, 4, 4$ para os itens $0, 1, 2, 3$. O item 0 fica na posição 0 e os demais empatam, resolvidos pelo identificador em ordem crescente. Os pontos de posição são, portanto, $4, 3, 2, 1$ para os itens $0, 1, 2, 3$.

Ordenando por SINERGIA, o Machado Genérico coincide exatamente com a build e vale 1. O Elmo do Primo compartilha o atributo força e vale $10/(\sqrt{8} \cdot 5) \approx 0{,}707$. A Barba Postiça e as Botas não tocam em força e valem 0. A ordenação é $0, 3, 1, 2$, com pontos de posição $4, 3, 2, 1$ respectivamente, multiplicados pelo peso 3.

Somando, o item 0 vale $4 + 12 = 16$, o item 1 vale $3 + 6 = 9$, o item 2 vale $2 + 3 = 5$ e o item 3 vale $1 + 9 = 10$. O item 0 está equipado e é descartado. Restam, em ordem, os itens 3, 1 e 2: o Elmo do Primo é recomendado à frente da Barba Postiça, apesar de terem poder bruto igual, porque combina melhor com o que Blibblon já usa. Na segunda consulta, os itens 0 e 3 empatam em `8.00` e são desempatados pelo identificador.

1.6 Tipos abstratos de dados

Você deve implementar um programa que mantenha dinamicamente o catálogo de modificadores, as builds dos aventureiros, a fila de encomendas e as estruturas necessárias para responder às consultas especificadas. Para isso, é obrigatória a definição, implementação e validação dos seguintes TADs:

- **Atributo**: armazena as informações associadas a um atributo, incluindo seu identificador e nome.
- **Item**: armazena as informações associadas a um item, incluindo seu identificador e nome.
- **Aventureiro**: armazena as informações associadas a um aventureiro, incluindo seu identificador, nome e a build atualmente equipada.
- **Catálogo**: TAD suficientemente geral para representar a matriz esparsa de modificadores nas duas representações exigidas, permitindo a troca da representação interna durante a execução, preservando o estado lógico, isto é, sem perda de células. Deve oferecer, no mínimo: consulta e atribuição de uma célula, remoção de célula, percurso de uma linha e percurso de uma coluna. Deve também expor as operações necessárias ao cálculo das métricas, incluindo o produto interno entre duas linhas esparsas.
- **Pilha**: armazena a build do aventureiro, com as operações necessárias à operação de desequipar.
- **Fila**: armazena as encomendas da forja, com as operações necessárias ao processamento em ordem de chegada.

A escolha de como Pilha e Fila são implementadas internamente, por vetor ou por encadeamento, é livre, mas deve ser justificada na documentação e avaliada na análise experimental. As demais estruturas auxiliares são de implementação livre.
Faz parte do trabalho a definição das funcionalidades desses TADs, bem como sua integração correta. Além disso, o trabalho envolve a otimização dos algoritmos utilizados. Em particular, deve-se investigar o compromisso entre manter o vetor da build atualizado a cada operação de equipar e desequipar e recomputá-lo apenas quando uma consulta o exigir.

2 Análise Experimental

A análise experimental tem por objetivo avaliar o desempenho da solução implementada em diferentes cenários de uso da forja. Em particular, deseja-se investigar como o custo das operações varia em função do tamanho do catálogo de itens, do número de atributos, da densidade da matriz de modificadores, do tamanho médio das builds e da frequência relativa entre atualizações e consultas. Para isso, sugerimos os seguintes passos:

1. Compare experimentalmente, pelo menos, dois perfis de carga:

   - **Perfil com predominância de atualizações**: a entrada contém alta frequência de operações M, F, Z e D, em comparação com a quantidade de consultas;
   - **Perfil com predominância de consultas**: a entrada contém alta frequência de operações LI, LA, LH e Q, em comparação com a quantidade de atualizações.

2. Compare as duas representações obrigatórias da matriz, considerando separadamente o custo de atualização e o custo de consulta. Espera-se que as representações não vençam nos mesmos cenários, e a discussão deve explicitar em quais e por quê. Reporte também o custo da própria migração entre modos.

3. Compare a estratégia imediata e a estratégia sob demanda para manter o vetor da build. Reporte o tempo médio de Z e D e o tempo médio de LH e Q em cada estratégia, identificando o ponto em que uma passa a compensar a outra.

4. Compare pelo menos três algoritmos de ordenação na implementação da consulta Q, escolhidos entre os estudados na disciplina. A escolha deve ser justificada em função do que se sabe sobre os dados: cada consulta exige m ordenações completas do catálogo, e a ordem induzida por uma métrica muda pouco entre consultas consecutivas se poucas operações M ocorreram no intervalo. Discuta o que isso implica para algoritmos que se beneficiam de entrada quase ordenada.

5. Compare pelo menos duas estratégias para o produto interno entre linhas esparsas, usado em SINERGIA e no cálculo do vetor da build: acumular em um vetor denso de tamanho T, ou intercalar duas listas já ordenadas sem alocar esse vetor. Meça as duas e explique em que regime de densidade cada uma vence.

6. Projete o experimento variando parâmetros que afetam diretamente o custo dos algoritmos, entre eles: o número total de itens; o número total de atributos; a densidade da matriz de modificadores; o tamanho médio das builds; o comprimento médio da fila; os valores de n e m nas consultas de recomendação; e a frequência relativa entre as operações.

7. Execute os experimentos e avalie os resultados, indicando claramente os parâmetros escolhidos em cada configuração experimental. Sempre que possível, reporte medidas como: tempo médio de cada operação em cada modo de armazenamento; uso total de memória em cada modo; número de células percorridas por consulta; número de comparações realizadas por ordenação; e tempo total de execução por perfil de carga.

Em termos de implementação, o ideal é que essas medidas sejam parte do TAD e apenas informadas ao programa usuário do TAD quando pertinente. Isso vai demandar não apenas a instrumentação do código do TAD para coletar essas medidas como o armazenamento dos valores internamente no TAD. A interface do TAD deve também prover mecanismos de recuperar esses valores para fins da experimentação.
A análise experimental deve ser conduzida com cuidado, para que as medidas obtidas reflitam de fato o comportamento da implementação avaliada. Em particular, é importante evitar que diferenças de organização do código ou de gerenciamento de memória interfiram nos resultados, produzindo tempos que não estejam relacionados diretamente ao custo dos algoritmos. Alocações dinâmicas desnecessárias dentro dos laços medidos, por exemplo, contaminam o resultado.

Os itens acima constituem apenas sugestões para a condução da análise experimental. Os alunos têm liberdade para definir outros cenários, métricas e experimentos que considerem relevantes. No entanto, as escolhas realizadas devem ser justificadas e devem ser coerentes com o objetivo de avaliar o desempenho da solução proposta.

2.1 Análise de localidade de referência

A localidade de referência descreve a tendência de um programa acessar, em um curto intervalo de tempo, dados próximos na memória ou dados que foram acessados recentemente. Esse comportamento é importante porque a CPU utiliza mecanismos como caches e linhas de cache para acelerar acessos repetidos ou sequenciais à memória.
Neste trabalho, a análise de localidade de referência deve ser **qualitativa**. Não é necessário instrumentar contadores de cache nem produzir gráficos de posições de memória acessadas ao longo do tempo; espera-se uma discussão fundamentada sobre o padrão de acesso das principais operações nas duas representações.
Como ponto de partida: no modo T, percorrer a linha de um item significa varrer posições contíguas de um vetor, e parte dos próximos dados necessários pode já estar disponível em cache. No modo L, os mesmos elementos podem estar espalhados pela memória, e cada acesso indireto por ponteiro pode falhar. Isso não decide a questão sozinho, porque a inserção no meio de um vetor ordenado tem um custo que a lista cruzada não possui. A consulta Q é o melhor caso para observar esse conflito, pois percorre o catálogo inteiro uma vez por métrica antes de ordenar. Explique como o padrão de acesso de cada representação se manifesta nos tempos que você mediu.

3 Pontos Extras

- Estenda o TAD Catálogo para suportar uma terceira representação interna, comprimida por linha: um vetor contíguo com os valores, um vetor com os identificadores de atributo correspondentes, e um vetor de índices indicando onde começa cada linha. A operação

  ```
  E <modo>
  ```

  deve ser estendida para aceitar um novo modo, definido e documentado por você, preservando a migração sem perda de células.

  A implementação deve ser descrita na documentação e avaliada na análise experimental, com atenção especial ao contraste entre o custo de percurso e o custo de inserção. Não haverá VPL para essa funcionalidade, e o novo modo não aparecerá nos casos de teste oficiais. Contudo, a nova representação deve funcionar e pode estar sujeita a correção manual.

- Modifique seu programa para aceitar também uma nova consulta por faixa de valores, com o seguinte formato:

  ```
  LF <id_atributo> <valor_min> <valor_max>
  ```

  Essa consulta deve listar todos os itens cujo modificador naquele atributo está no intervalo fechado $[valor_min, valor_max]$, em ordem crescente de identificador de item:

```
LF <nome_atributo> item_1 <id_i1> <valor_1> ... item_M <id_iM> <valor_M>
```

ou LF <nome_atributo> VAZIO se nenhum item satisfizer a faixa. Itens sem célula naquele atributo têm modificador zero e devem ser considerados se zero pertencer à faixa.

Para suportar essa consulta eficientemente, a implementação deve localizar os extremos da faixa por **busca binária sobre um vetor ordenado**, e não por varredura linear do catálogo. A documentação deve explicar qual estrutura foi incorporada ao TAD, qual o custo assintótico da consulta por faixa e qual o custo de manter essa estrutura atualizada diante das operações M.

Um VPL específico será utilizado para avaliar essa funcionalidade. Os programas submetidos para o VPL principal e para o VPL de pontos extras devem ser os mesmos.

4 Como será feita a entrega

4.1 Submissão

A avaliação do TP1 compreende duas submissões e uma entrevista individual posterior à entrega:

VPL TP1: Submissão do código a ser realizada até **19/10/2026, 23:59** (o sistema vai ficar aberto madrugada adentro, mais para evitar problemas transientes de infraestrutura). Detalhes sobre a submissão do código são apresentados na Seção 4.3.

Relatório TP1: Arquivo PDF contendo a documentação do TP, incluindo a análise de complexidade e a análise experimental, conforme as instruções apresentadas neste documento, a ser submetido até **19/10/2026, 23:59** (o sistema vai ficar aberto madrugada adentro, mais para evitar problemas transientes de infraestrutura). Detalhes sobre a submissão do relatório são apresentados na Seção 4.2.

Entrevista Individual: Após a entrega do trabalho, cada aluno participará de uma breve entrevista individual e presencial sobre sua própria implementação. A entrevista tem como objetivo avaliar a compreensão do aluno sobre as estruturas de dados, algoritmos, decisões de implementação e funcionamento do código submetido. Durante a entrevista, o aluno deverá realizar ou descrever pequenas modificações em seu próprio código, conforme detalhado na seção de avaliação. As atividades poderão ser respondidas utilizando código em C/C++ ou pseudocódigo equivalente, não sendo necessário que o código produzido durante a entrevista compile.

A participação na entrevista faz parte da avaliação do trabalho. A nota obtida nessa etapa será utilizada como um fator multiplicativo sobre a parcela da nota correspondente ao código, conforme descrito na Seção 5.

4.2 Documentação

A documentação do trabalho deve ser produzida utilizando o **Overleaf**, em LaTeX, seguindo o **formato de artigos da AAAI**. Para isso, deve ser utilizado como base o template oficial da AAAI disponibilizado no Overleaf. O formato do template deve ser preservado, incluindo organização em duas colunas, tipografia, margens e estilo das referências bibliográficas.

A documentação deve ser escrita com a estrutura e o estilo de um **artigo científico**, apresentando de forma clara o problema tratado, as decisões de implementação, sua fundamentação e a avaliação da solução desenvolvida. Não se espera uma descrição linha a linha do código-fonte, mas sim uma explicação das principais decisões algorítmicas e de estruturas de dados adotadas.
A documentação deve conter as seguintes seções:

1. **Resumo (*Abstract*):** Apresentação breve do problema, da solução desenvolvida, das principais estruturas e estratégias utilizadas e dos principais resultados obtidos na análise experimental.

2. **Introdução:** Apresentação do contexto e do problema tratado pelo trabalho, seguida de uma visão geral da solução desenvolvida. A introdução deve permitir que o leitor compreenda a motivação do trabalho e quais são os principais desafios computacionais envolvidos.

3. **Trabalhos Relacionados:** Breve apresentação das principais fontes utilizadas como apoio para o desenvolvimento do trabalho. Podem ser incluídos artigos científicos, livros, documentação técnica ou outras fontes relevantes relacionadas às estruturas de dados, algoritmos ou estratégias utilizadas. Não se espera uma revisão extensa da literatura; em geral, **um ou dois parágrafos são suficientes**. As fontes mencionadas devem ser devidamente citadas e incluídas nas referências.

4. **Métodos:** Descrição da solução implementada, incluindo as principais estruturas de dados, tipos abstratos de dados (TADs), algoritmos e decisões de implementação. Devem ser apresentadas e justificadas as escolhas realizadas durante o desenvolvimento, incluindo, quando pertinente, estratégias de robustez, programação defensiva e tratamento de situações especiais.

5. **Análise de Complexidade:** Análise da complexidade de tempo e espaço das principais operações e algoritmos implementados, utilizando notação assintótica. A análise deve estar relacionada às estruturas e estratégias efetivamente utilizadas na implementação.

6. **Análise Experimental:** Apresentação da metodologia experimental utilizada para avaliar a implementação, dos experimentos realizados e dos resultados obtidos. Os resultados devem ser apresentados de maneira adequada, utilizando tabelas e/ou figuras, e acompanhados de uma discussão que explique seu comportamento e os relacione às características das estruturas de dados e algoritmos avaliados.

7. **Conclusão:** Síntese do trabalho realizado e dos principais resultados encontrados. Deve também apresentar as principais conclusões obtidas a partir das análises teórica e experimental e discutir brevemente o que foi aprendido durante o desenvolvimento.

8. **Agradecimentos e Declaração de Uso de IA Generativa (*Acknowledgments*):** Esta seção deve conter uma declaração explícita sobre a utilização ou não de ferramentas de inteligência artificial generativa durante o desenvolvimento do trabalho e da documentação. Caso tenham sido utilizadas, o aluno deve informar quais ferramentas foram utilizadas e, de forma breve, para quais finalidades. O uso dessas ferramentas não transfere responsabilidade pelo conteúdo produzido: o aluno continua integralmente responsável pelo código e pela documentação entregues.

9. **Referências:** Lista das fontes citadas ao longo do artigo, utilizando o formato bibliográfico definido pelo template da AAAI. Toda fonte utilizada e mencionada no texto deve ser devidamente referenciada.

A documentação deve apresentar o trabalho em termos funcionais e conceituais, com foco nos algoritmos, estruturas de dados, decisões de implementação, análise de complexidade e evidências experimentais.
Evite a descrição literal do código-fonte. Trechos de código devem ser utilizados apenas quando forem necessários para explicar uma decisão ou algoritmo particularmente relevante.
Figuras, tabelas e gráficos devem ser legíveis, possuir legendas adequadas e ser mencionados e discutidos no texto. Resultados experimentais não devem ser apresentados sem interpretação.
A documentação deverá ser entregue em formato `.pdf`, gerado a partir do projeto no Overleaf, conforme as instruções de submissão disponibilizadas no `minha.ufmg`.

4.3 Código

Você deve utilizar a linguagem C ou C++ para o desenvolvimento do seu sistema. O uso de estruturas pré-implementadas pelas bibliotecas-padrão da linguagem ou terceiros é terminantemente vetado. Você DEVE utilizar a estrutura de projeto abaixo junto ao Makefile:

```
– TP
    |– src
    |– bin
    |– obj
    |– include
    Makefile
```

A pasta **TP** é a raiz do projeto; **src** deve armazenar arquivos de código (*.c, *.cpp, ou *.cc); a pasta **include**, os cabeçalhos (headers) do projeto, com extensão *.h, por fim as pastas **bin** e **obj** devem estar vazias. O Makefile deve estar na raiz do projeto. A execução do Makefile deve gerar os códigos objeto *.o no diretório **obj** e o executável do TP no diretório **bin**. O arquivo executável **DEVE** se chamar **tp1.out** e deve estar localizado na pasta **bin**. O código será compilado com o comando:

```
make all
```

O seu código será avaliado através de uma **VPL** que será disponibilizada no moodle. Você também terá à disposição uma VPL de testes para verificar se a formatação da sua saída está de acordo com a requisitada. A VPL de testes não vale pontos nem conta como trabalho entregue. Um pdf com instruções de como enviar seu trabalho para que ele seja compilado corretamente estará disponível no Moodle.

4.4 Entrevista Individual

Após a entrega do trabalho, cada aluno realizará uma **entrevista individual e presencial** utilizando como base o código submetido no VPL. O objetivo da entrevista é avaliar a compreensão do aluno sobre sua própria implementação, incluindo as estruturas de dados utilizadas, a lógica dos algoritmos, a organização do código e as decisões tomadas durante o desenvolvimento.

A avaliação será realizada de maneira progressiva, a partir de três níveis de questão: **difícil**, **média** e **fácil**. O aluno iniciará sempre pela questão difícil. Caso não consiga resolvê-la de maneira satisfatória, será apresentada a questão de nível médio. Caso também não consiga resolver a questão média, será apresentada a questão de nível fácil.
Cada questão será projetada para poder ser respondida em aproximadamente **5 a 10 minutos**. As respostas poderão ser apresentadas diretamente em **C/C++** ou utilizando **pseudocódigo equivalente**. O código produzido durante a entrevista não precisa compilar. Serão avaliadas principalmente a lógica apresentada, a capacidade de compreender e modificar a própria implementação e a justificativa das decisões tomadas.
Os níveis da entrevista serão definidos da seguinte forma:

Questão difícil: O aluno deverá propor e implementar uma pequena nova funcionalidade a partir do código originalmente submetido. A funcionalidade será simples e localizada, mas exigirá que o aluno compreenda como os componentes de sua implementação se relacionam e seja capaz de estendê-los corretamente. O desempenho nessa questão corresponde a um fator de entrevista no intervalo de **0,85 a 1,10**.

Questão média: O avaliador removerá previamente um pequeno trecho da implementação submetida pelo aluno. Durante a entrevista, o aluno deverá reconstruir esse trecho, preservando a lógica e o comportamento esperado do programa. O trecho selecionado será simples e diretamente relacionado a funcionalidades já existentes no trabalho. O desempenho nessa questão corresponde a um fator de entrevista no intervalo de **0,60 a 0,85**.

Questão fácil: O aluno deverá realizar ou descrever uma modificação pontual e simples em seu próprio código. A questão será elaborada de forma a exigir apenas a compreensão direta de uma parte específica da implementação. O desempenho nessa questão corresponde a um fator de entrevista no intervalo de **0,00 a 0,60**.

O valor obtido na entrevista será denominado **fator de entrevista** e pertencerá ao intervalo de **0,00 a 1,10**. Esse fator será utilizado como multiplicador da parcela da nota correspondente ao código avaliado pelo VPL, conforme detalhado na Seção 5.
A entrevista é individual e deverá refletir exclusivamente a compreensão do aluno sobre o trabalho submetido. Não será permitido consultar materiais externos, ferramentas de inteligência artificial, outros alunos ou versões alternativas do código durante a realização da entrevista.

5 Avaliação

A nota do trabalho será composta por duas parcelas: a avaliação do **código**, correspondente a **75% da nota total**, e a avaliação da **documentação**, correspondente aos **25% restantes**.

- **Código e VPL – 75% da nota total:** o código submetido será avaliado automaticamente por meio do VPL, resultando em uma nota entre 0 e 100. Essa nota será posteriormente ajustada pelo desempenho do aluno na entrevista individual, conforme descrito na Seção 4.4.
- **Documentação – 25% da nota total:** a documentação será avaliada considerando a qualidade e clareza do texto, a descrição e justificativa das decisões de implementação, a análise de complexidade, a metodologia e análise experimental,

a apresentação e discussão dos resultados e a aderência ao formato e às instruções estabelecidas na Seção 4.2.

5.1 Cálculo da nota

Seja:

- V a nota obtida no VPL, no intervalo $[0, 100]$;
- E o fator obtido na entrevista individual, no intervalo $[0, 1.10]$;
- D a nota da documentação, no intervalo $[0, 100]$.

A nota correspondente ao código será calculada por:

$$N_{\text{código}} = 0{,}75 \times V \times E.$$

A parcela correspondente à documentação será:

$$N_{\text{documentação}} = 0{,}25 \times D.$$

Assim, a nota final do trabalho será calculada por:

$$\boxed{N_{\text{final}} = 0{,}75 \times V \times E + 0{,}25 \times D}$$

O fator da entrevista atua, portanto, exclusivamente sobre a parcela correspondente ao código. Um fator igual a 1,00 mantém integralmente a nota obtida no VPL; fatores inferiores a 1,00 reduzem essa parcela; e fatores superiores a 1,00, até o máximo de 1,10, permitem bonificação para alunos que demonstrarem domínio particularmente elevado de sua implementação.
Por exemplo, um aluno que obtenha nota 80 no VPL, fator 0,90 na entrevista e nota 90 na documentação terá:

$$N_{\text{final}} = 0{,}75 \times 80 \times 0{,}90 + 0{,}25 \times 90 = 76{,}5.$$

5.2 Condição mínima no VPL

Para que o trabalho seja avaliado nas demais etapas, o aluno deverá obter **nota estritamente maior que zero no VPL**.
Caso a nota obtida no VPL seja igual a zero, o trabalho será considerado não funcional para fins de avaliação e receberá **nota final igual a zero**, não sendo avaliadas a entrevista individual ou a documentação.
Essa condição também se aplica a programas que não possam ser compilados ou executados corretamente pelo ambiente oficial de avaliação.
Trabalhos entregues com atraso sofrerão **penalização de 2^{d-1} pontos**, com d correspondendo ao número de dias úteis de atraso.

6 Considerações finais

1. Comece a fazer esse trabalho prático o quanto antes, enquanto o prazo de entrega está tão distante quanto jamais estará.

2. Leia atentamente o documento de especificação, pois o descumprimento de quaisquer requisitos obrigatórios aqui descritos causará penalizações na nota final.

3. Certifique-se de garantir que seu arquivo foi submetido corretamente no sistema.

4. Plágio é crime. Trabalhos onde o plágio for identificado serão **automaticamente anulados** e as medidas administrativas cabíveis serão tomadas (em relação a todos os envolvidos). Discussões sobre o trabalho entre colegas são permitidas. É permitido consultar fontes externas, desde que exclusivamente para fins didáticos e devidamente registradas na seção de bibliografia da documentação. **Cópia e compartilhamento de código não são permitidos.**

5. A utilização de ferramentas de inteligência artificial é de sua responsabilidade, ou seja, quaisquer erros, plágios ou dificuldade de entendimento associada a conteúdo generativo será devidamente penalizada.

7 FAQ (*Frequently asked Questions*)

1. Posso utilizar qualquer versão do C++? NÃO, o corretor da VPL utiliza C++11.

2. Posso fazer o trabalho no Windows, Linux, ou MacOS? SIM, porém lembre-se que a correção é feita sob o sistema Linux, então certifique-se que seu trabalho está funcional em Linux.

3. Posso utilizar alguma estrutura de dados do C++ do tipo Queue, Stack, Vector, List, etc? NÃO.

4. Posso utilizar smart pointers? NÃO.

5. Posso utilizar o tipo String? SIM.

6. Posso utilizar o tipo String para simular minhas estruturas de dados? NÃO.

7. Posso utilizar alguma biblioteca para tratar exceções? SIM.

8. Posso utilizar alguma biblioteca para gerenciar memória? SIM.

9. As análises e apresentação dos resultados são importantes na documentação? SIM.

10. Os meus princípios de programação ligados a C++ e relacionados a engenharia de software serão avaliados? NÃO.

11. Posso fazer o trabalho em dupla ou em grupo? NÃO.

12. Posso trocar informações com os colegas sobre os fundamentos teóricos do trabalho? SIM.

13. Posso utilizar IDEs, Visual Studio, Code Blocks, Visual Code, Eclipse? SIM.